

DMIT2015 – Lesson 4A Student Practice Activity

Java Review + Unit Testing Reinforcement

Overview

In this activity, you will:

- Create a Java class using IntelliJ IDEA
- Add constructor validation
- Create JUnit test methods
- Use AssertJ assertions
- Run and verify automated tests

This activity reinforces:

- Java syntax
- packages and classes
- IntelliJ workflow
- JUnit testing
- AssertJ assertions

Learning Outcomes

By completing this activity, you should be able to:

- Create Java classes in a package
- Write constructor validation logic
- Create and run JUnit tests
- Use AssertJ assertions
- Apply Arrange-Act-Assert structure

Part 1 – Create the Product Class

Step 1 – Create Package

Create the following package:

```
dmitt2015.model
```

```
}
```

Step 2 – Create Product.java

Create `Product.java` under `src/main/java/dmit2015/model`. Implement the following requirements yourself, using the instructor's Book example as a guide.

- Define private fields `String name` and `double price`.
- Create a constructor with the signature `public Product(String name, double price)`.
- Reject a null, empty, or whitespace-only name by throwing `IllegalArgumentException` with the message `Product name is required`.
- Reject a price less than or equal to zero by throwing `IllegalArgumentException` with the message `Price must be greater than 0`.
- Store valid constructor arguments in the fields.
- Provide `getName()` and `getPrice()` getters.
- Implement `public double getPriceWithTax()` to return the price including 5% tax: `price * 1.05`.

Before coding, identify valid inputs, invalid inputs, and boundary values. Decide how to check the name safely when it may be null.

Spend approximately 15 minutes developing your class and tests before comparing your approach with a classmate. Use the remaining 5 minutes to discuss differences and troubleshoot.

Part 2 — Create `ProductTest.java`

Step 1 — Create Test Class

Create the following package if it does not already exist:

```
src/test/java/dmit2015/model
```

▢

Create:

```
ProductTest.java
```

▢

Step 2 — Add Required Imports

```
import org.junit.jupiter.api.Test;
```

```
import static org.assertj.core.api.Assertions.*;
```

▢

Part 3 — Create Test Methods

Write your own test methods from the scenarios below. Use descriptive names and Arrange-Act-Assert. You may use separate test methods for each invalid input; parameterized tests are not required.

Test 1 — Verify Product Name

Create a product with name "Laptop" and price 1200. Verify that `getName()` returns "Laptop".

Test 2 – Verify Product Price

Using the same valid constructor inputs, verify that `getPrice()` returns 1200. Together, Tests 1 and 2 verify valid object creation.

Test 3 – Verify Tax Calculation

Calculate the expected total independently for a price of 1200 and 5% tax. Verify that `getPriceWithTax()` returns 1260, using a tolerance of 0.01 for the double comparison.

Test 4 – Verify Invalid Product Name

Test each of these names with a valid price of 100:

- `null`
- `""`
- `" "`

For each input, verify that the constructor throws `IllegalArgumentException` and that the message contains `required`.

Test 5 – Verify Invalid Price

Test both 0 and -5 with a valid name of "Laptop". For each input, verify that the constructor throws `IllegalArgumentException` and that the message contains `greater than 0`.

Guided Checkpoints

- Use `assertThat(...)` to check returned values.
- Use `isCloseTo(..., within(0.01))` for double comparisons.
- Use `assertThatThrownBy(() -> ...)` for constructor validation.
- Keep the other constructor argument valid when testing one invalid input.
- Explain why testing zero matters in addition to testing a negative price.

Part 4 – Run the Tests

Instructions

Run:

`ProductTest.java`



Verify:

- All tests pass
- IntelliJ displays a green test result bar
- Every required scenario above is covered

Temporarily change one expected result, run the test, and explain the failure. Restore the correct expectation and rerun all tests.

Bonus Challenge (Optional)

Add:

```
private int quantity;
```

▢

Then:

- update constructor
- create getter
- create:

```
getInventoryValue()
```

▢

Formula:

```
price * quantity
```

▢

Create a corresponding unit test.

Submission

No Brightspace submission required unless instructed by your instructor.

Be prepared to:

- demonstrate your code
- explain your tests
- show successful test execution